



MVSR ENGINEERING COLLEGE

(An Autonomous Institution)

(Sponsored by Matrusri Educational Society, Est: 1981)

Affiliated to Osmania University | Accredited by NAAC & NBA

Nadargul, Hyderabad - 501510



Department of Computer Science & Engineering

LAB RECORD MANUAL

Subject: FULL STACK DEVELOPMENT LAB

Code: U21PC582CT

Branch: CSIT

Class: B.E- V SEM

Acad.Year: 2025-26

*Prepared By:
P. Phani Prasad,
Asst. Professor,
CSED, MVSREC*

TABLE OF CONTENTS

Sl.no	Name of the Lab Experiment	Date of Conduct
PART-I Working with React JS		
1.1	ReactJS components	18.08.2025
1.2	ReactJS JSX	23.08.2025
1.3	Styling of React JS	25.08.2025
1.4	ReactJS props and state	13.09.2025
1.5	Conditional and List Rendering	22.09.2025
1.6	ReactJS Events	27.09.2025
1.7	React JS Hooks: useState and useContext	06.10.2025
PART-II Working with Node JS		
2.1	Built in Module : prompt-sync	13.10.2025
2.2	Built in Module : fs – Creating and Writing data to a file	13.10.2025
2.3	Built in Module : fs – Reading data from a file	13.10.2025
2.4	Built in Module : fs – Appending data to a file	13.10.2025
2.5	Built in Module : http	25.10.2025
2.6	Built in Module : url	25.10.2025
PART-III Working with MongoDB		
3.1	Perform CRUD operations using MongoDB SHELL	27.10.2025
3.2	Perform CRUD operations using MongoDB Compass	27.10.2025
PART-IV Node JS and MongoDB		
4.1	Inserting a Student record	01.11.2025
4.2	Fetching and Displaying Student records	01.11.2025
4.3	Updating Student records	08.11.2025
4.4	Deleting student records	08.11.2025

PART-V Working with Express JS		
5.1	Loading an html page	10.11.2025
5.2	Creating multiple routes and perform page navigation	10.11.2025
5.3	Arithmetic Operations using Form Handling	15.11.2025
5.4	Demonstrate on Template Engine:EJS	15.11.2025
PART-VI Working with Python Flask		
6.1	Create multiple routes and perform page navigation	16.11.2025
6.2	Loading static files html file,css and javascript files into flask	16.11.2025
6.3	Handling forms through html methods	17.11.2025
6.4	Demonstrate on Template Engine:Jinja	22.11.2025
PART-VII CRUD operations using MongoDB and Flask		
7.1	Inserting employee record in MongoDB using flask	24.11.2025
7.2	Fetching and displaying employee records in MongoDB using flask	24.11.2025
7.3	Updating employee record in MongoDB using flask	29.11.2025
7.4	Deleting employee record in mongoDB using flask	29.11.2025

PART- I Working with React JS

React JS: It is a built-in javascript library for building rich look & feel attractive Front - End User Interfaces in web development.

It is based on Components and designed by Meta in 2013.

1.1 Working with React JS Components

AIM: Write a React JS program to demonstrate React JS components

PROCESS:

- A) Create a new Folder “Components” and open it using VS code.
- B) Open Terminal in VS Code and type below command
npx create-react-app component-demo
- C) After above command completion type below command
cd component-demo
- D) Now,in VSCode right click on src folder and create below Javascript files
 - a) FunctionDemo.js b) ClassCompDemo.js
- E) Edit [App.js](#) and import the above two files into it.
- F) Save all the above files and Run the Application using below command in the terminal
npm start

PROGRAM:

- a) FunctionDemo.js

```
function FunctionDemo()  
{  
    return <h1>Function Component Executed</h1>  
}  
export default FunctionDemo;
```

- a) ClassCompDemo.js

```
import { Component } from "react";
class ClassComDemo extends Component
{
  render()
  {
    return <h1>Class Component Executed</h1>
  }
}
export default ClassComDemo;
```

b) App.js

```
import './App.css';
import FunctionDemo from './FunctionDemo';
import ClassComDemo from './ClassCompDemo';
function App() {
  return (
    <div className="App">
      <h1>ReactJS Application to demonstrate Components</h1>
      <hr/>
      <FunctionDemo/>
      <ClassComDemo/>
    </div>
  );
}
export default App;
```

OUTPUT:

ReactJS Application to demonstrate Components

Function Component Executed

Class Component Executed

1.2 Working with React JS JSX

AIM: Write a React JS program to demonstrate React JS JSX

PROCESS:

- A) Create a new Folder “reactdemos” and open it using VS code.
- B) Open Terminal in VS Code and type below command
npx create-react-app jsx-demo
- C) After above command completion type below command
cd jsx-demo
- D) Now,in VSCode right click on src folder and create below Javascript files
 - b) WithJsx.js b) WithoutJsx.js
- E) Edit [App.js](#) and import the above two files into it.
- F) Save all the above files and Run the Application using below command in the terminal
npm start

PROGRAM:

- a) WithJsx.js

```
import React from "react";
function WithJsx()
{
    return(<div align="center">
        <h1>This is H1 tag executed using JSX</h1>
        <p>This is paragraph tag executed using JSX</p>
    </div>)
}
export default WithJsx;
```

- b) WithoutJsx.js

```
import React from "react";
function WithoutJsx()
{
    let k1=React.createElement("h1",null,"This is H1 tag executed without JSX")
    let k2=React.createElement("p",null,"This is Paragraph tag executed without JSX")
    let k3=React.createElement("div",{align:"center"},k1,k2);
    return k3;
}
export default WithoutJsx;
```

- c) App.js

```
import WithJsx from './WithJsx';
import WithoutJsx from './WithoutJsx';
function App() {
  return (
    <div align="center">
      <h3>React JS Program to demonstrate JSX</h3><hr/>
      <WithJsx/>
      <WithoutJsx/>
    </div>
  );
}
export default App;
```

OUTPUT:

React JS Program to demonstrate JSX

This is H1 tag executed using JSX

This is paragraph tag executed using JSX

This is H1 tag executed without JSX

This is Paragraph tag executed without JSX

1.3 Styling of React JS

AIM: To develop a ReactJS Application that demonstrates CSS Styling on React JS components .

PROCESS:

- A) Create a new Folder “Styling” and open it using VS code.
- B) Open Terminal in VS Code and type below command
npx create-react-app styling-demo
- C) After above command completion type below command
cd styling-demo
- D) Now,in VSCode right click on the src folder and create a new folder “styling”.
- E) Create below Javascript files and CSS files.
 - a) Sample.css b) Rules.module.css c) StyleDemo.js
- F) Edit App.js and import the above StyleDemo.js file into it.
- G) Save all the above files and Run the Application using below command in the terminal
npm start

PROGRAM:

- a) Sample.css

```
p
{
    color:  blue;
    border: 5px solid  green;
    background-color:  bisque;
    font-style: italic;
    font-size: 40px;
    margin-left: 400px;
    margin-right: 400px;
}
```

- b) Rules.module.css

```
.rule1
```

```
{
```

```
    color:  white;
```

```
    background-color:  darkmagenta;
```

```
    border: 10px dotted  yellow;
```

```
    margin-left: 300px;
```

```
    margin-right: 300px;
```

```
    font-size: 40px;
```

```
}
```

c) StyleDemo.js

```
import "./Sample.css"
import styles from "./Rules.module.css"
function StyleDemo()
{
  return(
    <div>
      <h1 style={{color:"red",fontSize:"30px"}}>
        This is H1 tag with Inline Style
      </h1>
      <p>This is paragraph with external style</p>
      <h2 className={styles.rule1}>
        This is H2 tag with external Module style
      </h2>
    </div>
  );
}
export default StyleDemo;
```

OUTPUT:

This is H1 tag with Inline Style

This is paragraph with external style

This is H2 tag with external Module style

1.4 React JS props and state

AIM: To develop a ReactJS Application that demonstrates props and state.

PROCESS:

- A) Create a new Folder “PropsDemos” and open it using VS code.
- B) Open Terminal in VS Code and type below command
npx create-react-app prop-state-demo
- C) After above command completion type below command
cd prop-state-demo
- D) Now, in VSCode right click on src folder and create below Javascript files
a) PropDemo.js b) StateDemo.js
- E) Edit [App.js](#) and import the above two files into it.
- F) Save all the above files and Run the Application using below command in the terminal
npm start

PROGRAM:

- a) Empprops.js

```
function Empprops(props)
{
    return(
        <div>
        <h3>Employee details using props are:</h3>
        <hr/>
        <p>Name:{props.name}<br/>
        Eid:{props.eid}<br/>
        Salary:{props.salary}
        </p>
        </div>
    );
}
export default Empprops;
```

b) Studentcomp.js

```
import { Component } from "react";
class Studentcomp extends Component
{
  constructor(props)
  {
    super(props);
    this.state={rno:123,name:"smith",marks:70,branch:"CSE-DS"}
  }
  change=()=>
  {
    this.setState({rno:345,name:"SMITH",marks:80,branch:"CSE-DS"})
  }
  render()
  {
    return(
      <div>
        <h3>Student Details using State</h3><hr/>
        <p>Roll number:{this.state.rno}<br/>
          Name:{this.state.name}<br/>
          Branch:{this.state.branch}<br/>
          Marks:{this.state.marks}<br/>
        </p>
        <button onClick={this.change}>Click to change</button>
      </div>
    );
  }
}
export default Studentcomp;
```

c) App.js

```
import './App.css';
import Empprops from './propsdemo/Empprops';
import Studentcomp from './propsdemo/Studentcomp';
function App() {
  return (
    <div className="App">
      <Empprops name="John" eid={123} salary={6000}/>
      <Studentcomp/>
    </div>
  );
}
export default App;
```

OUTPUT:

Employee details using props are:	Employee details using props are:
Name:John Eid:123 Salary:6000	Name:John Eid:123 Salary:6000
Student Details using State	Student Details using State
Roll number:123 Name:smith Branch:CSE-DS Marks:70 Click to change	Roll number:345 Name:SMITH Branch:CSE-DS Marks:80 Click to change

1.5 Conditional and List Rendering

AIM: To develop a ReactJS Application that demonstrates Conditional Rendering and List Rendering

PROCESS:

- A) Create a new Folder “RenderingDemos” and open it using VS code.
- B) Open Terminal in VS Code and type below command
npx create-react-app rendering-demo
- C) After above command completion type below command
cd rendering-demo
- D) Now, in VSCode right click on src folder and create two Javascript files, “[Conditional.js](#)” and “List.js”
- E) Edit App.js and import the above two files into it.
- F) Save all the above files and Run the Application using below command in the terminal
npm start

PROGRAM:

- a) Conditional.js

```
function Conditional(props)
{
    let k=props.value;
    if(k)
        return <h1>True Conditional tag executed</h1>
    else
        return <h1>False Conditional tag executed</h1>
}
export default Conditional;
```

- b) List.js

```
function List()
{
    let items=["CSE", "CSIT", "DS", "IOT", "AIML"]
    let temp=[]
    for(let k=0;k<items.length;k++)
        temp.push(<li>{items[k]}</li>)
    return (<>
        <h3>Displaying List using for loop</h3><br/>
        <ol>{temp}</ol>
    )
}
```

```

    <hr/>
    <h3>Displaying List using map function</h3><br/>
    <ul>
      {
        items.map((value)=><li>{value}</li>)
      }
    </ul>
  </>)
}
export default List;

```

c) App.js

```

import Conditional from "../Conditional";
import List from "../List";
function App() {
  return (
    <div align="center">
      <h3>React JS Program to demonstrate Conditional
Rendering and List Rendering</h3><hr/>
      <Conditional value={true}/>
      <Conditional value={false}/>
      <List/>
    </div>
  );
}
export default App;

```

OUTPUT:

True Conditional tag executed

False Conditional tag executed

Displaying List using for loop

1.
2.
3.
4.
5.

CSE
CSIT
DS
IOT
AIML

Displaying List using map function

•
•
•
•
•

CSE
CSIT
DS
IOT
AIML

1.6 React JS Events

AIM: To develop a ReactJS Application that performs Arithmetic operations using Events.

PROCESS:

- A) Create a new Folder “EventDemos” and open it using VS code.
- B) Open Terminal in VS Code and type below command
npx create-react-app event-demo
- C) After above command completion type below command
cd event-demo
- D) Now,in VSCode right click on src folder and create a Javascript file, “Arithmetic.js”
- E) Edit App.js and import the above Arithmetic.js file into it.
- F) Save all the above files and Run the Application using below command in the terminal
npm start

PROGRAM:

- d) Arithmetic.js

```
import { Component } from "react";
class Arithmetic extends Component
{
  constructor(props)
  {
    super(props);
    this.state={a:0,b:0,c:""}
  }
  changeA=(event)=>{this.setState({a:event.target.value})}
  changeB=(event)=>{this.setState({b:event.target.value})}
  add=()=>{
    let x=parseInt(this.state.a);
    let y=parseInt(this.state.b);
    this.setState({c:(x+y)})
  }
  sub=()=>{
    let x=parseInt(this.state.a);
    let y=parseInt(this.state.b);
    this.setState({c:(x-y)})
  }
  mul=()=>{
    let x=parseInt(this.state.a);
```

```
        let y=parseInt(this.state.b);
        this.setState({c:(x*y)})
    }
    div={()=>{
        let x=parseInt(this.state.a);
        let y=parseInt(this.state.b);
        this.setState({c:(x/y)})
    }}
    render()
    {
        return(
            <div>
                <h4>Arithmetic Operations using React
Events</h4><hr/>
                Enter first number:
                <input type="text" value={this.state.a}
onChange={this.changeA}/>
                <br/>
                Enter second number:
                <input type="text" value={this.state.b}
onChange={this.changeB}/>
                <br/>
                <button
onClick={this.add}>Add</button>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
                <button
onClick={this.sub}>Subtract</button>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
                <button
onClick={this.mul}>Multiply</button>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
                <button onClick={this.div}>Divide</button><br/>
                Result={this.state.c}
            </div>
        );
    }
}
```

```
export default Arithmetic;
```

e) App.js

```
import './App.css';
import Arithmetic from './Arithmetic';
function App() {
  return (
    <div className="App">
      <Arithmetic/>
    </div>
  );
}
export default App;
```

OUTPUT:

Arithmetic Operations using React Events Enter first number: 50 Enter second number: 5 Add Subtract Multiply Divide Result=55	Arithmetic Operations using React Events Enter first number: 50 Enter second number: 5 Add Subtract Multiply Divide Result=45
Arithmetic Operations using React Events Enter first number: 50 Enter second number: 5 Add Subtract Multiply Divide Result=250	Arithmetic Operations using React Events Enter first number: 50 Enter second number: 5 Add Subtract Multiply Divide Result=10

1.7 React JS Hooks: useState and useContext

1.7.1 Demonstrate on useState Hook

AIM: To develop a ReactJS Application that demonstrates useState Hooks.

PROCESS:

- Create a new Folder “HookDemo” and open it using VS code.
- Open Terminal in VS Code and type below command
npx create-react-app usestate-demo
- After above command completion type below command
cd usestate-demo
- Now, in VSCode right click on src folder and create a Javascript file, “UseStateDemo.js”
- Edit App.js and import the above UseStateDemo.js file into it.
- Save all the above files and Run the Application using below command in the terminal
npm start

PROGRAM:

a) UseStateDemo.js

```
import { useState } from "react"

function UseStateDemo()

{
    let [count,setCount]=useState(0);
    const increase=()=>setCount(++count)
    const decrease=()=>setCount(--count)
    return(<div align='center'>
        <button onClick={increase}>Increment</button>
        &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
        ~~~~~
        <button onClick={decrease}>Decrement</button>
    </div>)
}

export default UseStateDemo
```

b) App.js

```
import UseStateDemo from "./UseStateDemo";
function App() {
  return (
    <div align="center">
      <UseStateDemo/>
    </div>
  );
}
```

```
    </div>
  );
}
export default App;
```

OUTPUT:

Increment5Decrement	Increment3Decrement
--------------------------------	--------------------------------

1.7.2 React JS Hooks: useContext

AIM: To develop a ReactJS Application that demonstrates useContext Hook.

PROCESS:

- A) Create a new Folder “HookDemo2” and open it using VS code.
- B) Open Terminal in VS Code and type below command
npx create-react-app usecontext-demo
- C) After above command completion type below command
cd usecontext-demo
- D) Now, in VSCode right click on src folder and create Three Javascript files: “[Parent.js](#)”, “[Child.js](#)” and “GrandChild.js”
- E) Edit App.js and import the above files wherever required as per below programs.
- F) Save all the above files and Run the Application using below command in the terminal
npm start

PROGRAM:

a) Parent.js

```
import Child from "../Child"
function Parent()
{
    return (<>
        <h3>Parent Component</h3>
        <Child/>
    </>)
}
export default Parent;
```

b) Child.js

```
import GrandChild from "../GrandChild"
function Child()
{
    return (<>
        <h3>Child component</h3>
        <GrandChild/>
    </>)
}
export default Child;
```

c) GrandChild.js

```

import { UserNameContext } from "../App";
import { useContext } from "react";
function GrandChild()
{
    let data=useContext(UserNameContext);
    return(<>
        <h3>Grand Child component</h3>
        <h3>Data Inherited from Main Application={data}</h3>
    </>)
}
export default GrandChild;

```

d) App.js

```

import Parent from "../Parent"
import React from "react";
export const UserNameContext=React.createContext();
function App() {
    return (
        <div align="center">
            <UserNameContext.Provider value={"MVSR"}>
                <Parent/>
            </UserNameContext.Provider>
        </div>
    );
}
export default App;

```

OUTPUT:

Parent Component

Child component

Grand Child component

Data Inherited from Main Application=MVSR

PART-II Working with Node JS

Node JS: It is an open source, cross platform, javascript runtime environment to build server-side web applications.

It is used as a both front-end and back-end server side scripting language to process clients requests and generate responses.

It can interact with databases like mongodb.

It is used for building Single Page Applications (SPA's)

It is rich in built-in modules and thus can be used in many various applications like Cryptography, blockchain, web apps, mobile applications etc.

2.1 Built in modules: prompt-sync

It is a built-in module that allows the user to read string data from the terminal.

AIM: To design a node js application that reads a string and prints its reverse.

PROCESS:

- A) Create a new folder “nodejsdemo” and open it using VS Code IDE
- B) Create a new file “stringreverse.js” and type the following program.
- C) Open terminal in VS Code IDE and run the below command to get the output
node stringreverse

PROGRAM:

```
const prompt=require("prompt-sync") ()
let str=prompt("Enter a string:")
console.log("Given string =" +str)
let k1=str.split("")
let k2=k1.reverse()
let k3=k2.join("")
console.log("Reversed string="+k3)
```

OUTPUT:

```
> node index
Enter a string:abcd
Given string =abcd
Reversed string=dcba
```

2.2 Built in modules: fs – creating and writing data to file

“fs” means File System.

It is a built-in module that allows the user to file I/O operations such as creating, writing, reading, appending data to a file, renaming and deleting files.

AIM: To design a node js application that creates a file and writes some data to the file.

PROCESS:

- A) Create a new folder “nodejsdemo” and open it using VS Code IDE
- B) Create a new file “filewrite.js” and type the following program.
- C) Open terminal in VS Code IDE and run the below command to get the output
node filewrite

PROGRAM:

```
const file=require("fs")
const prompt=require("prompt-sync")()
let fname=prompt("Enter file name to create:")
let data=prompt("Enter some text:")
file.writeFileSync(fname,data)
console.log("File created and data written to file")
```

OUTPUT:

```
> node filewrite
Enter file name to create:sample.txt
Enter some text:Hello World
File created and data written to file
```

```
> cat sample.txt
Hello World
```

2.3 Built-in module: fs – Reading data from a file

AIM: To design a node js application that opens a file and reads the contents of the file.

PROCESS:

- A) Create a new folder “nodejsdemo” and open it using VS Code IDE
- B) Create a new file “fileread.js” and type the following program.
- C) Open terminal in VS Code IDE and run the below command to get the output
node fileread

PROGRAM:

```
const file=require("fs")
const prompt=require("prompt-sync")()
let fname=prompt("Enter file name:")
let data=file.readFileSync(fname,"utf-8")
console.log("File Contents are:\n=====")
console.log(data)
```

OUTPUT:

```
> node fileread
Enter file name:sample.txt
File Contents are:
=====
Hello World
```

2.4 Built-in module: fs – Appending data to the file

AIM: To design a node js application that opens a file and appends some data to the file.

PROCESS:

- A) Create a new folder “nodejsdemo” and open it using VS Code IDE
- B) Create a new file “filewrite.js” and type the following program.
- C) Open terminal in VS Code IDE and run the below command to get the output
node fileappend

PROGRAM:

```
const file=require("fs")
const prompt=require("prompt-sync")()
let fname=prompt("Enter file name to open:")
let data=prompt("Enter some text:")
file.appendFileSync(fname,data)
console.log("Data appended to file")
```

OUTPUT:

<pre>> cat sample.txt Hello World</pre>	<pre>> node fileappend Enter file name to open:sample.txt Enter some text: MVSR College Data appended to file</pre>	<pre>> cat sample.txt Hello World MVSR College</pre>
--	--	---

2.5 Built-in module: http

“Http” module is a built-in module in node js used to implement a two tier client-server model. It creates a server object to receive clients requests via browser url, process it and generates dynamic responses in different MIME formats like plain text,image,pdf,audio,video etc.

AIM: To design a node js application that demonstrates “http” module

PROCESS:

- A) Create a new folder “nodejsdemo” and open it using VS Code IDE
- B) Create a new file “httpdemo.js” and type the following program.
- C) Open terminal in VS Code IDE and run the below command to start http server
node httpdemo
- D) Open browser and type localhost:3000, press enter to get the output

PROGRAM:

```
const http=require("http")
const server=http.createServer((req,res)=>
{
    res.writeHead(200,{"Content-Type":"text/html"})
    res.end("<h1>Http Module executed</h1>")
})
server.listen(3000,()=>
{
    console.log("server started and running on port 3000")
})
```

OUTPUT:

```
> node httpdemo.js
server started and running on port 3000
```

Http Module executed

2.6 Built-in module: url

“Url” module: It is a built-in module in node js used to parse a given website URL and fetch url information from the given website.

AIM: To design a node js application that demonstrates the “url” module.

PROCESS:

- A) Create a new folder “nodejsdemo” and open it using VS Code IDE
- B) Create a new file “urldemo.js” and type the following program.
- C) Open terminal in VS Code IDE and run the below command to get the output
node urldemo

PROGRAM:

```
const { URL } = require('url');
const
str="https://user:abcd@www.example.com:4000/abc/xyz/sample.txt?
value1=10&value2=20#demo"
const myUrl = new URL(str);
console.log("Hostname="+myUrl.hostname);
console.log("Portnumber="+myUrl.port);
console.log("Relative path="+myUrl.pathname);
console.log("Queryparams="+myUrl.search);
console.log("Hashed data="+myUrl.hash);
console.log("Username="+myUrl.username);
console.log("Password="+myUrl.password);
console.log("Protocol="+myUrl.protocol);
```

OUTPUT:

```
> node urldemo.js
```

```
Hostname=www.example.com
```

```
Portnumber=4000
```

```
Relative path=/abc/xyz/sample.txt
```

```
Queryparams=?value1=10&value2=20
```

```
Hashed data=#demo
```

```
Username=user
```

```
Password=abcd
```

```
Protocol=https:
```

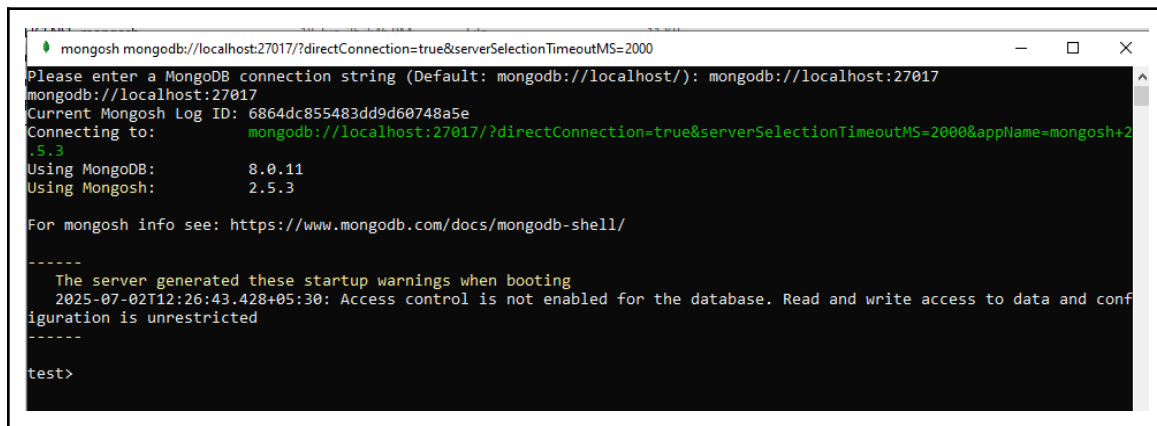
PART-III Working with MongoDB

3.1 Perform Basic CRUD operations using MongoDB SHELL

AIM: To run MongodB Shell and perform simple CRUD operations

PROCEDURE (Using Windows OS):

- In the Windows Operating System, Open Command Prompt and Run as Administrator.
- Start MongoDB Server using below command
net start MongoDB
- GoTo MongodB shell directory and open mongosh.exe file.
- Type the below Connection URL and press enter
mongodb://localhost:27017
- Now we can operate MongoDB Commands



```
mongosh mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000
Please enter a MongoDB connection string (Default: mongodb://localhost/): mongodb://localhost:27017
mongodb://localhost:27017
Current Mongosh Log ID: 6864dc855483dd9d60748a5e
Connecting to:  mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.3
Using MongoDB:      8.0.11
Using Mongosh:      2.5.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2025-07-02T12:26:43.428+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test>
```

PROCEDURE (Using Ubuntu OS):

- In the Ubuntu Operating System, Open Terminal and type the command below to start MongoDB Server. Enter user login password if it asks.
sudo service mongod start
- Type below command to start MongodB shell
mongosh
- We will see the below screen after typing the above command.

```

mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ sudo service mongod start
[sudo] password for mvsr:
mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mongosh
Current Mongosh Log ID: 69185d163d2153903032a03b
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=
2000&appName=mongosh+2.5.5
Using MongoDB:      4.4.1
Using Mongosh:      2.5.5
Mongosh 2.5.9 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2025-11-15T21:56:22.480+05:30: ***** SERVER RESTARTED *****
2025-11-15T21:56:22.550+05:30: Using the XFS filesystem is strongly recommended with the WiredT
iger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2025-11-15T21:56:22.942+05:30: Access control is not enabled for the database. Read and write a
ccess to data and configuration is unrestricted
2025-11-15T21:56:22.942+05:30: You are running this process as the root user, which is not reco
mmended
2025-11-15T21:56:22.943+05:30: Soft rlimits too low
-----
test>

```

CRUD OPERATIONS

a) Creating a “employee” Collection

```
db.createCollection("employee")
```

```
test> db.createCollection("employee");
{ ok: 1 }
```

b) Creating a single “employee” document

```
db.employee.insertOne({"_id":123,"ename":"John","salary":5000,"age":35,"gender":"male"});
```

```
test> db.employee.insertOne({"_id":123,"ename":"John","salary":5000,"age":35,"gender":"male"});
{ acknowledged: true, insertedId: 123 }
```

c) Creating multiple “employee” documents

```
db.employee.insertMany([{"_id":124,"ename":"Mary","salary":7000,"age":45,"gender":"female"}, {"_id":125,"ename":"Angel","salary":2000,"age":20,"gender":"female"}]);
```

```
test> db.employee.insertMany([{"_id":124,"ename":"Mary","salary":7000,"age":45,"gender":"female"}, {"_id":125,"ename":"Angel","salary":2000,"age":20,"gender":"female"}]);
{ acknowledged: true, insertedIds: { '0': 124, '1': 125 } }
```

d) Fetching “employee” documents

db.employee.find();	<pre>test> db.employee.find(); [{ _id: 123, ename: 'John', salary: 5000, age: 35, gender: 'male' }, { _id: 124, ename: 'Mary', salary: 7000, age: 45, gender: 'female' }, { _id: 125, ename: 'Angel', salary: 2000, age: 20, gender: 'female' }] test></pre>
---------------------	--

e) Fetching specific “employee” document

db.employee.find({"_id":123});	<pre>test> db.employee.find({"_id":123}); [{ _id: 123, ename: 'John', salary: 5000, age: 35, gender: 'male' }] test></pre>
--------------------------------	--

f) Updating a specific “employee” document

db.employee.updateOne({"ename":"John"},{\$set:{"salary":15000}});	<pre>test> db.employee.updateOne({"ename":"John"},{\$set:{"salary":15000}}); { acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 } test></pre>
---	--

g) Updating multiple “employee” documents

db.employee.updateMany({"age":{\$gt:20}},{\$set:{"salary":20000}});	<pre>test> db.employee.updateMany({"age":{\$gt:20}},{\$set:{"salary":20000}}); { acknowledged: true, insertedId: null, matchedCount: 2, modifiedCount: 2, upsertedCount: 0 } test></pre>
---	--

h) Deleting a specific “employee” document

db.employee.deleteOne({"_id":125})	<pre>test> db.employee.deleteOne({"_id":125}) { acknowledged: true, deletedCount: 1 } test></pre>
------------------------------------	---

i) Deleting all “employee” documents

db.employee.deleteMany({});	<pre>test> db.employee.deleteMany({}); { acknowledged: true, deletedCount: 2 } test></pre>
-----------------------------	--

3.2 Performing basic CRUD operations using MongoDB Compass

AIM: To run MongoDB Compass and perform simple CRUD operations

PROCESS (Using Windows OS):

- In the Windows Operating System, Open Command Prompt and Run as Administrator.
- Start MongoDB Server using below command
net start MongoDB
- On the Desktop, double click MongoDB Compass Icon to open Compass tool
- Click on “Add new connection”. Ensure the connection string is
mongodb://localhost:27017
- Click the “Save & Connect” button. Now we can perform CRUD operations.

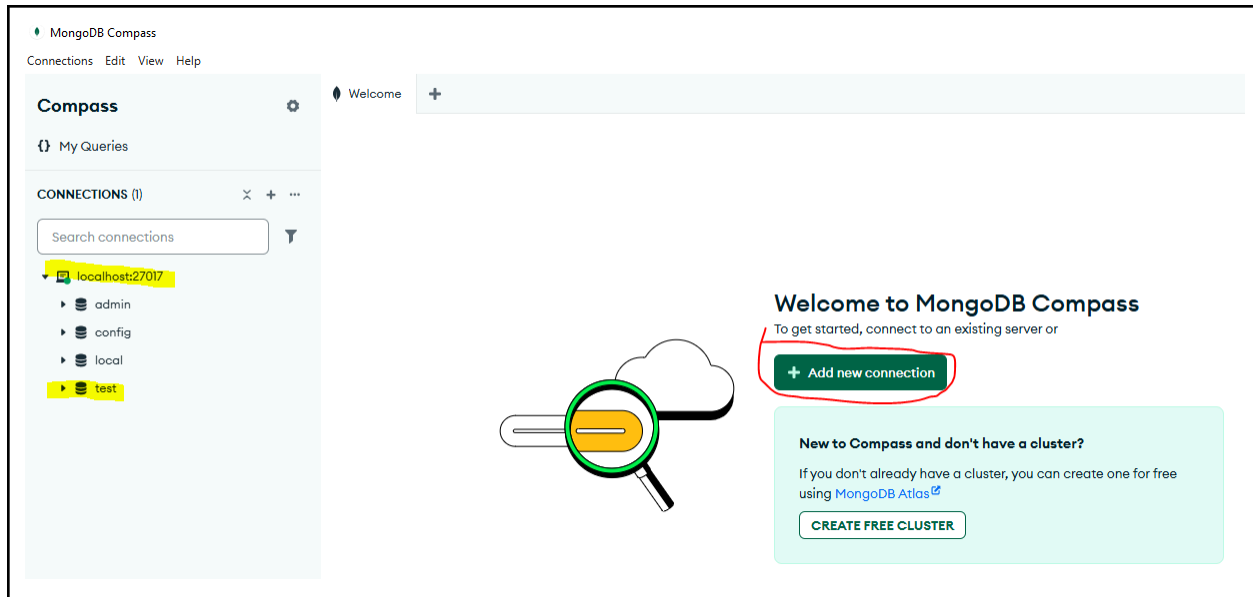
PROCESS (Using Ubuntu OS):

- In the Ubuntu Operating System, Open Terminal and type the command below to start MongoDB Server. Enter user login password if it asks.
sudo service mongod start
- Type below command to start MongoDB shell
mongosh
- We will see the below screen after typing the above command.

```
mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ sudo service mongod start
[sudo] password for mvsr:
mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mongosh
Current Mongosh Log ID: 69185d163d2153903032a03b
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=
2000&appName=mongosh+2.5.5
Using MongoDB:      4.4.1
Using Mongosh:      2.5.5
mongosh 2.5.9 is available for download: https://www.mongodb.com/try/download/shell

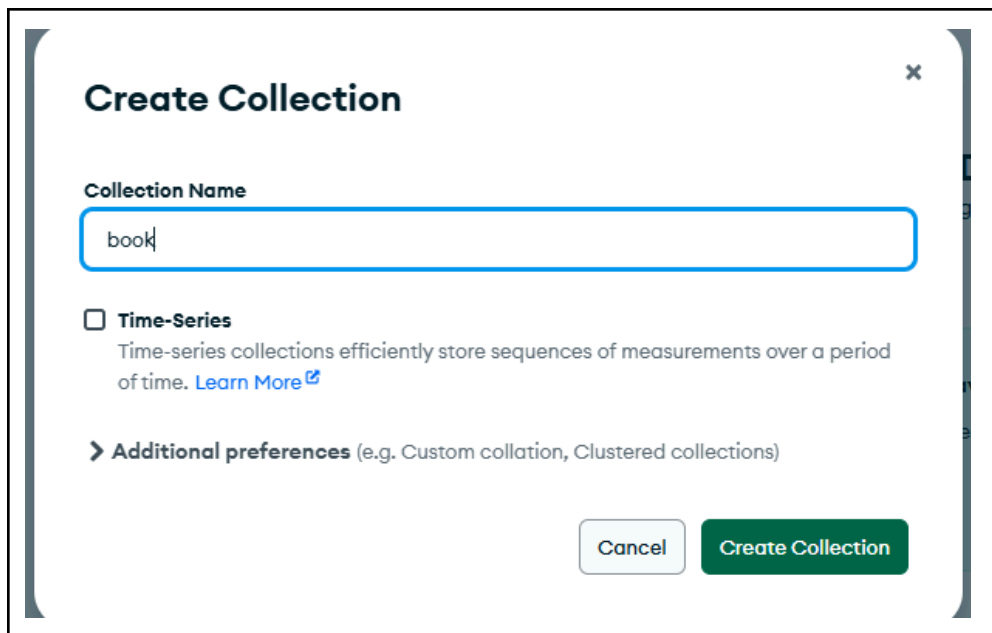
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2025-11-15T21:56:22.480+05:30: ***** SERVER RESTARTED *****
  2025-11-15T21:56:22.550+05:30: Using the XFS filesystem is strongly recommended with the WiredT
iger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
  2025-11-15T21:56:22.942+05:30: Access control is not enabled for the database. Read and write a
ccess to data and configuration is unrestricted
  2025-11-15T21:56:22.942+05:30: You are running this process as the root user, which is not reco
mmended
  2025-11-15T21:56:22.943+05:30: Soft rlimits too low
-----
test> |
```

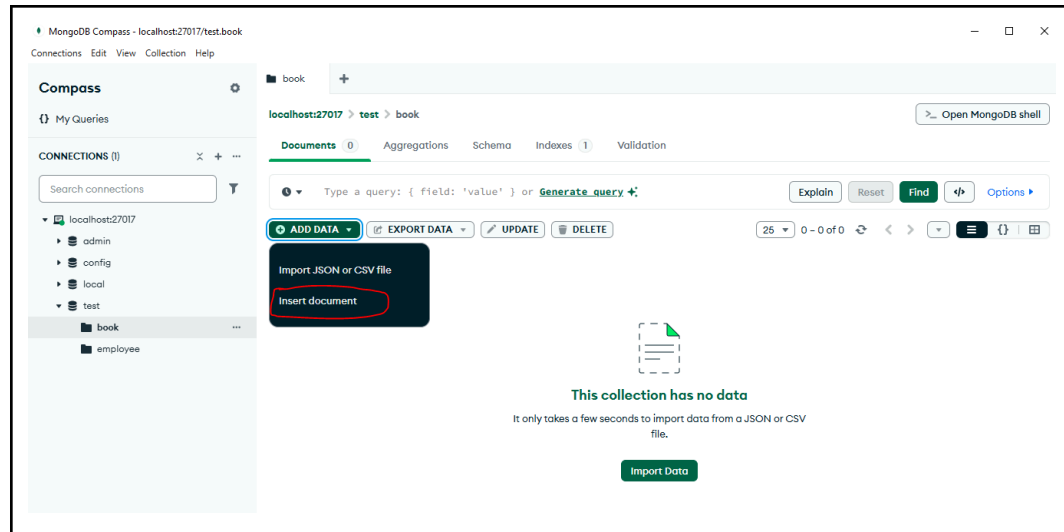


CRUD OPERATIONS

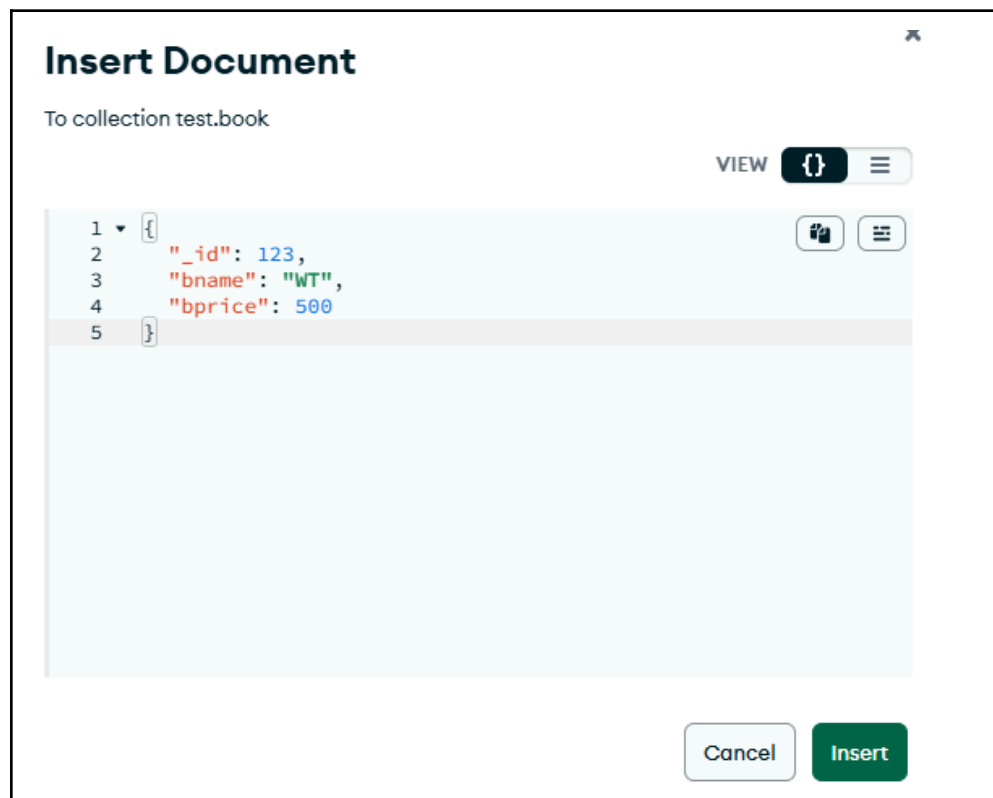
- a) Creating a new Collection.
Click the + symbol beside the test.
Give collection name (For example:book) and click on “Create Collection” Button



- b) Add Data to Collection
 - i) Click on “Add Data” and then select “Insert Document”



- ii) Write the data as key:value pairs as shown below and click “insert”. A document will be inserted.



- iii) Repeat the above step for inserting a few more documents.

+ ADD DATA
 EXPORT DATA
UPDATE
DELETE

▶
 `_id: 123
 bname: "WT"
 bprice: 500`

`_id: 124
 bname: "Html"
 bprice: 600`

`_id: 125
 bname: "Java"
 bprice: 800`

`_id: 126
 bname: "NodeJS"
 bprice: 1200`

c) Fetching Documents

- To fetch all documents ,click on “Find” button
- To fetch a specific document, type key:value pair in the textbox above the “Add Data” button and press the “Find” button. For example,to fetch book whose id is 123, it is shown below

⌵

Generate query
Explain
Reset
Find

+ ADD DATA
 EXPORT DATA
UPDATE
DELETE
25
1 - 1 of 1
↺
↻
↷

`_id: 123
 bname: "WT"
 bprice: 500`

d) Updating Documents

- In the textbox, give the key:value pair and click on the “Update” button. For example,to update the price whose id=123,it is shown below.

PART-IV Node JS and MongoDB

Node JS: It is an open source, cross platform and javascript builtin framework for running server side applications.

It lets the users run applications outside of the browser, typically on a server.

It can be used for many real time applications such as

- Building web servers and APIs
- Real-time applications (Live scores,chatting apps, multiplayer games)
- Command-line tools
- Streaming services
- Microservices

Key differences between React JS and Node JS is React JS used for Front-End User Interfaces and Node JS is used for Server side scripting and Backend applications

MongoDB: It is a NoSQL, document-oriented database system that stores data in JSON-like documents.

Unlike SQL databases (MySQL, PostgreSQL), MongoDB does not require fixed tables or schemas.

Ideal for applications where data structures can change frequently

Steps to Start MongoDB Server (Using Windows OS):

- a) In the Windows Operating System,Open Command Prompt and Run as Administrator.
- b) Start MongoDB Server using below command
net start MongoDB
- c) On the Desktop, double click MongoDB Compass Icon to open Compass tool
- d) Click on “Add new connection”. Ensure the connection string is
mongodb://localhost:27017
- e) Click the “Save & Connect” button. Now we can perform CRUD operations.

Steps to Start MongoDB Server (Using Ubuntu OS):

- a) In the Ubuntu Operating System,Open Terminal and type the command below to start MongoDB Server. Enter user login password if it asks.
sudo service mongod start
- b) Type below command to start Mongoddb shell
mongosh
- c) We will see the below screen after typing the above command.

```

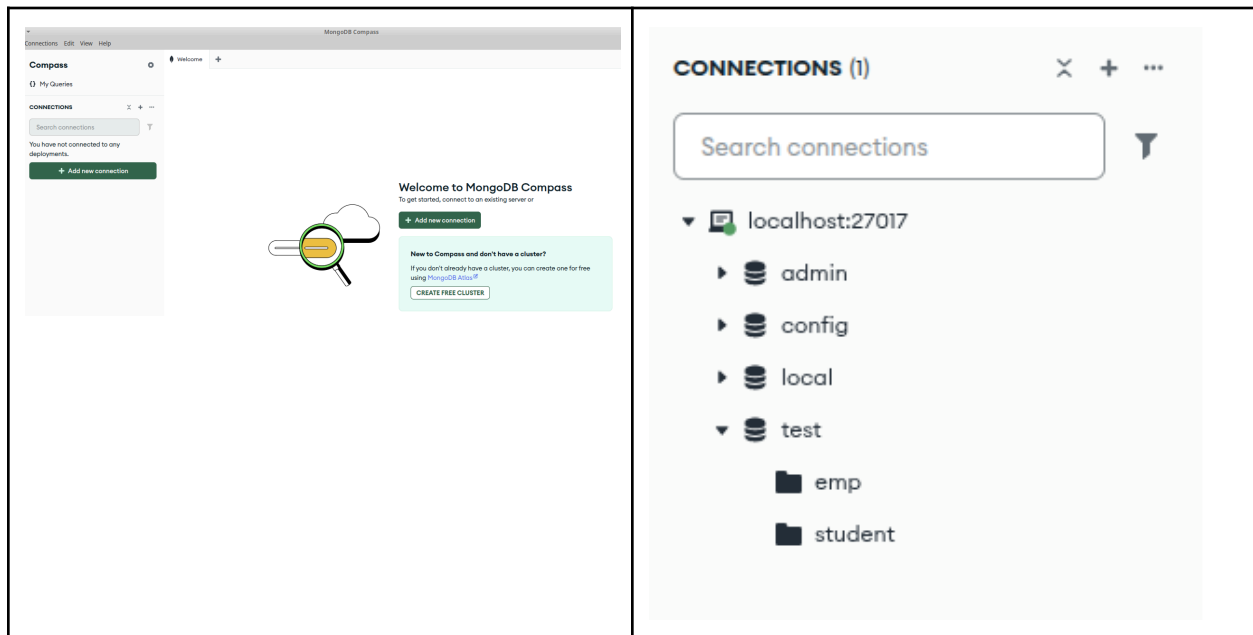
mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ sudo service mongod start
[sudo] password for mvsr:
mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ mongosh
Current Mongosh Log ID: 69185d163d2153903032a03b
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=
2000&appName=mongosh+2.5.5
Using MongoDB:      4.4.1
Using Mongosh:      2.5.5
mongosh 2.5.9 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2025-11-15T21:56:22.480+05:30: ***** SERVER RESTARTED *****
2025-11-15T21:56:22.550+05:30: Using the XFS filesystem is strongly recommended with the WiredT
iger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2025-11-15T21:56:22.942+05:30: Access control is not enabled for the database. Read and write a
ccess to data and configuration is unrestricted
2025-11-15T21:56:22.942+05:30: You are running this process as the root user, which is not reco
mmended
2025-11-15T21:56:22.943+05:30: Soft rlimits too low
-----
test>

```

Open MongoDB Compass



4.1 Inserting a Student Record

AIM: To write a Node JS program that inserts few Student Records in the MondoDB database.

PROCESS:

- a) Start the MongoDB Server
- b) Open Mongoddb shell
- c) Use test database and Create a student collection using below command
db.createCollection("student")
- d) Open MongoDB Compass and connect to student collection
- e) Create a new folder "NodeJsDemos" and open it in VS Code IDE
- f) Create a new file "StudentInsert.js" and write below program
- g) Open Terminal and type below command to run the program
node StudentInsert
- h) In the MongoDB compass, check records are reflected or not.

PROGRAM:

StudentInsert.js

```
const mdb=require("mongodb")
const prompt=require("prompt-sync") ()
const mclient=mdb.MongoClient
const url="mongodb://localhost:27017"
async function connectDB()
{
    try {
        const conn=await mclient.connect(url);
        console.log("Mongodb connection success...")
        let n=prompt("How many records you want to insert?")
        n=parseInt(n)
        const test=conn.db("test")
        const std=test.collection("student")
        let query=[]
        for(let i=0;i<n;i++)
        {
            console.log(`Enter details of ${i} record:\n=====\n `)
            let id=parseInt(prompt("enter id:"));
            let name=prompt("enter name:");
            let marks=parseInt(prompt("enter marks:"))
            let branch=prompt("enter branch:")
            let city=prompt("enter city:")
            let record={
                "_id":id,"name":name,"marks":marks,"branch":branch,"city":city
```

```

    }
    query.push(record)
  }
  await std.insertMany(query);
  console.log(n+" records inserted")
  conn.close();
} catch (error) {
  console.log("Error="+error)
}
}
connectDB()

```

OUTPUT:

```

mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/fsdlab$ node StudentInsert
Mongodb connection success...
How many records you want to insert?2
Enter details of 0 record:
=====
enter id:121
enter name:John
enter marks:70
enter branch:CSE
enter city:Hyd
Enter details of 1 record:
=====
enter id:122
enter name:Smith
enter marks:80
enter branch:CSIT
enter city:Delhi
2 records inserted

```

ConnectionsEditViewCollectionHelp

Compass

My Queries

CONNECTIONS (1) × + ...

Search connecti

localhost:27017

- admin
- config
- local
- test
 - emp
 - student

student

localhost:27017 > test > student

Documents 1AggregationsSchemaIndexes 1Validation

Type a query: { field: 'value' } or Get

ExplainResetFind

+ ▾

🔗 ▾✎🗑

25 ▾1 - 2 of 2 ↺⏪⏩

_id: 121

name: "John"

marks: 70

branch: "CSE"

city: "Hyd"

▶_id: 122

name: "Smith"

marks: 80

branch: "CSIT"

city: "Delhi"

4.2 Fetching and Displaying Student Records

AIM: To write a Node JS program that fetches and displays Student Records in the MondoDB database.

PROCESS:

- Start the MongoDB Server
- Open MongoDB shell
- Use test database and verify student collection exists or not using below command
db.student.find({})
- Open MongoDB Compass and connect to student collection
- Create a new folder "NodeJsDemos" and open it in VS Code IDE
- Create a new file "StudentFetch.js" and write below program
- Open Terminal and type below command to run the program
node StudentFetch

PROGRAM:

StudentFetch.js

```
const mdb=require("mongodb")
const prompt=require("prompt-sync")()
const mclient=mdb.MongoClient
const url="mongodb://localhost:27017"
async function connectDB()
{
    try {
        const conn=await mclient.connect(url);
        console.log("MongoDB connection success...")
        const test=conn.db("test")
        const std=test.collection("student")
        let result=await std.find({}).toArray()
        console.log(result);
        console.log("Number of records fetched="+result.length)
        conn.close()
    }catch(error)
    {
        console.log("Error="+error)
    }
}
connectDB()
```

OUTPUT:

```
mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/fsdlab$ node StudentFetch.js
Mongodb connection success...
[
  { _id: 121, name: 'John', marks: 70, branch: 'CSE', city: 'Hyd' },
  { _id: 122, name: 'Smith', marks: 80, branch: 'CSIT', city: 'Delhi' }
]
Number of records fetched=2
```

4.3 Updating Student Records using Node JS

AIM: To write a Node JS program that updates Student Records in the MondoDB database.

PROCESS:

- a) Start the MongoDB Server
- b) Open Mongoddb shell
- c) Use test database and verify student collection exists or not using below command
db.student.find({})
- d) Open MongoDB Compass and connect to student collection
- e) Create a new folder “NodeJsDemos” and open it in VS Code IDE
- f) Create a new file “StudentUpdate.js” and write below program
- g) Open Terminal and type below command to run the program
node StudentUpdate
- h) In the MongoDB Compass / Shell, check records are reflected or not.

PROGRAM:

StudentUpdate.js

```
const mdb=require("mongodb")
const prompt=require("prompt-sync") ()
const mclient=mdb.MongoClient
const url="mongodb://localhost:27017"
async function connectDB()
{
    try {
        const conn=await mclient.connect(url)
        const test=conn.db("test")
        const std=test.collection("student")
        console.log("Mongodb connection success...")
        let query1={},query2={}
        let skey=prompt("Enter search key:")
        let svalue=prompt("Enter search value:")
        query1[skey]=svalue
        let ukey=prompt("Enter update key:")
        let uvalue=parseInt(prompt("Enter updated value:"))
        query2[ukey]=uvalue
        await std.updateMany(query1,{ $set:query2 })
        conn.close()
        console.log("Record(s) updated")
    } catch (error) {
        console.log("Error="+error)
    }
}
```

```
}  
connectDB()
```

OUTPUT:

```
mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/fsdlab$ node StudentUpdate.js  
Mongodb connection success...  
Enter search key:name  
Enter search value:John  
Enter update key:marks  
Enter updated value:90  
Record(s) updated
```

```
test> db.createCollection("student");  
{ ok: 1 }  
test> db.student.find({})  
[  
  { _id: 121, name: 'John', marks: 90, branch: 'CSE', city: 'Hyd' },  
  { _id: 122, name: 'Smith', marks: 80, branch: 'CSIT', city: 'Delhi' }  
]  
test> █
```

4.4 Deleting Student Records

AIM: To write a Node JS program that deletes Student Records in the MondoDB database.

PROCESS:

- a) Start the MongoDB Server
- b) Open MongoDB shell
- c) Use test database and verify student collection exists or not using below command
db.student.find({})
- d) Open MongoDB Compass and connect to student collection
- e) Create a new folder “NodeJsDemos” and open it in VS Code IDE
- f) Create a new file “StudentDelete.js” and write below program
- g) Open Terminal and type below command to run the program
node StudentDelete
- h) In the MongoDB Compass / Shell, check records are reflected or not.

PROGRAM:

StudentDelete.js

```
const mdb=require("mongodb")
const prompt=require("prompt-sync") ()
const mclient=mdb.MongoClient
const url="mongodb://localhost:27017"
async function connectDB()
{
    try {
        const conn=await mclient.connect(url)
        const test=conn.db("test")
        const std=test.collection("student")
        console.log("Mongodb connection success...")
        let query1={}
        let skey=prompt("Enter search key:")
        let svalue=prompt("Enter search value:")
        query1[skey]=svalue
        await std.deleteMany(query1)
        conn.close()
        console.log("Record(s) Deleted")
    } catch (error) {
        console.log("Error="+error)
    }
}
connectDB()
```

OUTPUT:

```
test> db.createCollection("student");
{ ok: 1 }
test> db.student.find({})
[
  { _id: 121, name: 'John', marks: 90, branch: 'CSE', city: 'Hyd' },
  { _id: 122, name: 'Smith', marks: 80, branch: 'CSIT', city: 'Delhi' }
]
test> █
```

```
mvsr@mvsr-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~/Desktop/fsdlab$ node StudentDelete.js
Mongodb connection success...
Enter search key:name
Enter search value:John
Record(s) Deleted █
```

```
test> db.student.find({})
[
  { _id: 122, name: 'Smith', marks: 80, branch: 'CSIT', city: 'Delhi' }
]
test> █
```

PART-V Working with Express JS

Express JS: It is a minimal, flexible, fast and unopinionated web application framework for Node.js.

- It used to build client-server based web applications.
- It runs as a part of Node JS program to accept client requests, process data and generate dynamic html responses.
- It uses the concept of Routing and Template Engines to simplify the task of Dynamic Web content generation.
- To use express js ,we must install the module using below command

npm install express

5.1 Loading Html Page into Express JS

AIM: To design an express js application to load and display Html page

PROCESS:

- a) Create a new folder “ExpressDemos” and open it in VS Code IDE
- b) Create two new files “server.js” and “index.html” and write the below programs.
- c) Open Terminal and run the below command
node server
- d) Open browser and type below url to see the output
localhost:3000

PROGRAM:

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Express</title>
</head>
<body>
  <div align="center">
    <h3>Loading Html Page using Express JS</h3><hr/>
    <h2 style="color: red;">HTML Page Loaded Successfully...</h2>
  </div>
</body>
</html>
```

server.js

```
const express=require("express")
const file=require("fs")
const app=express()
let data=file.readFileSync('index.html',"utf-8")
app.get("/",(req,res)=>
{
    res.send(data)
})
app.listen(3000,()=>
{
    console.log("server started...")
})
```

OUTPUT:

```
PS D:\FSDLab> node server
server started...
```

Loading Html Page using Express JS

HTML Page Loaded Successfully...

5.2 Creating multiple routes and perform page navigation

AIM: To design an express js application to create multiple routes and perform page navigation

PROCESS:

- Create a new folder “ExpressDemos” and open it in VS Code IDE
- Create a new file “server2.js” and write the below program.
- Open Terminal and run the below command
node server2
- Open browser and type below url to see the output
localhost:3000

PROGRAM:

server2.js

```
const express=require("express")
const app=express()
app.get("/",(req,res)=>
{
    res.send("<h2 align='center'>This is Main Page<br/><a
href='/home'>Goto Home Page</a></h2>")
})
app.get("/home",(req,res)=>
{
    res.send("<h2 align='center'>This is Home Page<br/><a
href='/dashboard'>Goto Dashboard Page</a></h2>")
})
app.get("/dashboard",(req,res)=>
{
    res.send("<h2 align='center'>This is Dashboard Page<br/><a
href='/'>Goto Main Page</a></h2>")
})
app.listen(3000,()=>
{
    console.log("server started...")
})
```

OUTPUT:

<pre>PS D:\FSDLab> node server2 server started...</pre>	This is Main Page <u>Goto Home Page</u>
This is Home Page <u>Goto Dashboard Page</u>	This is Dashboard Page <u>Goto Main Page</u>

5.3 Arithmetic operations using Form Handling

AIM: To design an express js application to perform arithmetic operations using forms

PROCESS:

- Create a new folder "ExpressDemos" and open it in VS Code IDE
- Create two new files "index.html" and "arithmeticserver.js" and write the below program.
- Open Terminal and run the below command
node arithmeticserver
- Open browser and type below url to see the output
localhost:3000

PROGRAM:

Arithmetic.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Arithmetic</title>
</head>
<body>
  <div align="center">
    <h3>Arithmetic Operations using Express JS</h3><hr>
    <form action="/arithmetic" method="get">
      <label>Enter first number:</label>
      <input type="text" name="n1"/><br><br>
      <label>Enter second number:</label>
      <input type="text" name="n2"/><br><br>
      <label>Select an Operation:</label>
      <select name="op">
        <option value="add">Addition</option>
        <option value="sub">Subtraction</option>
        <option value="mul">Multiplication</option>
        <option value="div">Division</option>
      </select> <br><br>
      <input type="submit" value="Submit"/>
    </form>
  </div>
</body>
</html>
```

arithmeticserver.js

```
const express=require("express")
const file=require("fs")
let data=file.readFileSync("arithmetic.html","utf-8")
const app=express()
app.get("/",(req,res)=>
{
    res.send(data)
})
app.get("/arithmetic",(req,res)=>
{
    let a=parseInt(req.query.n1)
    let b=parseInt(req.query.n2)
    let op=req.query.op
    if(op=="add")
        res.send(`

### { console.log("server started...") })


```

OUTPUT:

<pre>PS D:\FSDLab> node arithmeticserver.js server started...</pre>	
Arithmetic Operations using Express JS Enter first number: 50 Enter second number: 20 Select an Operation: Addition ▼ Submit	First number=50 Second number=20 Addition=70
Arithmetic Operations using Express JS Enter first number: 50 Enter second number: 20 Select an Operation: Subtraction ▼ Submit	First number=50 Second number=20 Subtraction=30
Arithmetic Operations using Express JS Enter first number: 50 Enter second number: 20 Select an Operation: Multiplication ▼ Submit	First number=50 Second number=20 Product=1000

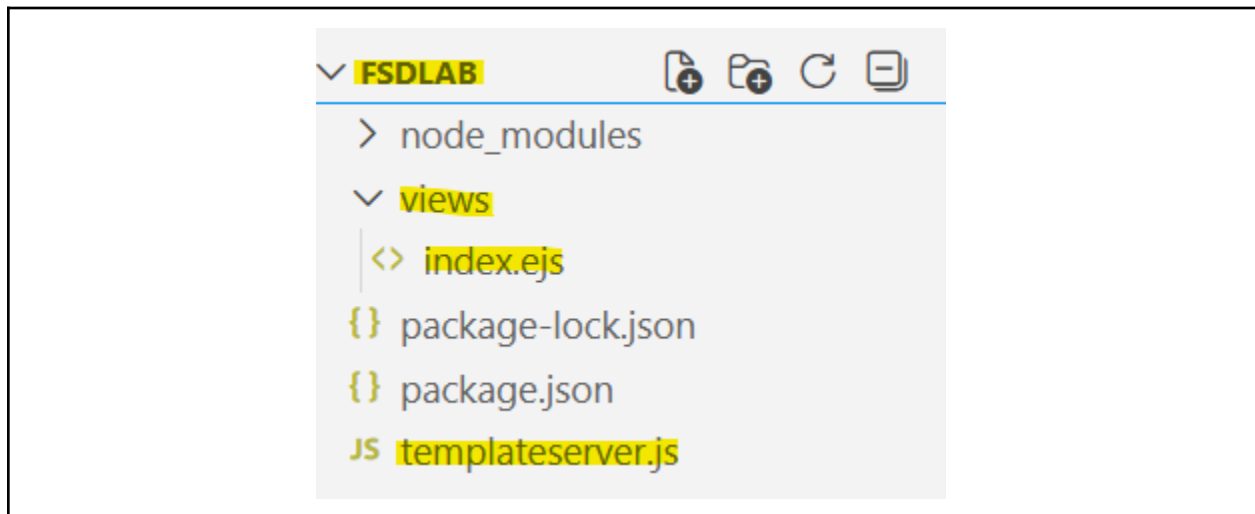
5.4 Demonstrate on Template Engine:EJS

AIM: To design an express js application to demonstrate Template Engine (EJS)

PROCESS:

- Create a new folder “ExpressDemos” and open it in VS Code IDE
- Open Terminal and type below commands to install modules
npm install express ejs
- Create a new file “templateserver.js” and write the below program.
- Create a folder “views”
- Create a new file “index.ejs”. Write the below code and save it in the “views” folder.
- Open Terminal and run the below command
node templateserver
- Open browser and type below url to see the output
localhost:3000

Folder Structure:



PROGRAM:

templateserver.js

```
const express=require("express")
const app=express()
app.set("view engine","ejs")
let
data={"eid":1234,"name":"John","job":"Clerk","salary":7000,"address":"De
lhi"}
app.get("/",(req,res)=>
{
    res.render("index",data)
})
```

```
app.listen(3000, () =>
{
    console.log("server started...")
})
```

index.ejs

```
<!DOCTYPE html>
<html>
<head>
    <title>EJS Demo</title>
</head>
<body>
    <div align="center">
        <h3>Example of Template Engine Using EJS</h3><hr>
        <h4>Displaying Employee Details</h4>
        <h4>Employee ID=<%=eid%></h4>
        <h4>Employee Name=<%=name%></h4>
        <h4>Employee Salary=<%=salary%></h4>
        <h4>Employee Job=<%=job%></h4>
        <h4>Employee Address=<%=address%></h4>
    </div>
</body>
</html>
```

OUTPUT:

```
PS D:\FSDLab> node templateserver.js
server started...
```

Example of Template Engine Using EJS

Displaying Employee Details

Employee ID=1234

Employee Name=John

Employee Salary=7000

Employee Job=Clerk

Employee Address=Delhi

PART - VI Working with Python Flask

Python Flask: Flask is an open source, cross platform and lightweight PYTHON based Framework for designing web applications.

6.1 Create multiple routes and perform page navigation

AIM: To design a flask application to create multiple routes and link pages

PROCESS:

- a) Create a folder “flaskdemos” and open it in VS Code IDE
- b) Write the below python code and save it as “routes.py”
- c) Open terminal and type below command to start Flask Server
 - i) For Ubuntu
python3 routes.py
 - ii) For Windows
python routes.py
- d) Open browser and type below Url to see the output
localhost:5000

PROGRAM:

```
from flask import Flask
app=Flask(__name__)
@app.route('/')
def helloWorld():
    return "<h3>Hello World.You Executed Flask</h3><a href='/page1'>Goto Page 1</a>"
@app.route('/page1')
def display1():
    return "<h3>This is Page 1 Display</h3><a href='/page2'>Goto Page 2</a>"
@app.route('/page2')
def display2():
    return "<h3>This is Page 2 Display</h3><a href='/'>Goto Main</a>"

if __name__ == '__main__':
    app.run()
```

OUTPUT:

```
D:\PythonFlask>python routes.py
* Serving Flask app 'routes'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [30/Nov/2025 23:23:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/Nov/2025 23:23:49] "GET /page1 HTTP/1.1" 200 -
127.0.0.1 - - [30/Nov/2025 23:24:29] "GET /page2 HTTP/1.1" 200 -
```

← → ↻ ⓘ localhost:5000

Hello World.You Executed Flask

[Goto Page 1](#)

← → ↻ ⓘ localhost:5000/page1

This is Page 1 Display

[Goto Page 2](#)

← → ↻ ⓘ localhost:5000/page2

This is Page 2 Display

[Goto Main](#)

6.2 Loading of static files: html,css,javascript

AIM: To design a flask application that loads html, css and javascript files into flask

PROCESS:

This application requires four files:

- i) index.html
- ii) style.css
- iii) script.js
- iv) load.py

Arrange above files as per below steps.

- a) Create a folder “flaskdemos” and open it in VS Code IDE
- b) Write the below python code and save it as “load.py” and save it in “flaskdemos” folder
- c) Create two folders “templates” and “static” inside “flaskdemos”
- d) Create a new file “index.html” and save it in the “templates” folder.
- e) Create a new file “style.css” and save it in the “static” folder.
- f) Create a new file “script.js” and save it in the “static” folder.
- g) Write below codes and save them.
- h) Open terminal and type below command to start Flask Server
 - i) For Ubuntu
python3 routes.py
 - ii) For Windows
python routes.py
- i) Open browser and type below Url to see the output
localhost:5000

PROGRAM:

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Static Files</title>
    <link rel="stylesheet"
href="{{url_for('static',filename='style.css')}}" />
</head>
<body>
    <div>
        <h1>Example of Static Files Loading</h1><hr>
    </div>
    <p id="p1"></p>
    <br/><br/>
    <script src="{{url_for('static',filename='demo.js')}}"></script>
    <button onclick="message('MVSR') ">Click</button>
</body>
</html>
```

script.js

```
function message(user)
{
    let k="Hello "+user+",Javascript loaded"
    document.getElementById("p1").innerHTML=k
}
```

style.css

```
div{
    background-color: brown;
    border: 5px solid greenyellow;
    font-size: x-large;
    margin-right: 800px;
}
h1{
    font-size: x-large;
    font-weight: bold;
    color: floralwhite
}
p{
    color: blue;
    font-size: xx-large;
    font-style: italic;
    font-weight: bold;
}
```

load.py

```
from flask import Flask,render_template
app=Flask(__name__)
@app.route("/")
def hello():
    return render_template("index.html")
if __name__=="__main__":
    app.run()
```

OUTPUT:

```
D:\PythonFlask>python load.py
* Serving Flask app 'load'
* Debug mode: off
WARNING: This is a development server. Do not use it in a
.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [30/Nov/2025 23:40:48] "GET / HTTP/1.1" 200
127.0.0.1 - - [30/Nov/2025 23:40:49] "GET /static/style.css HTTP/1.1" 200
127.0.0.1 - - [30/Nov/2025 23:40:49] "GET /static/demo.js HTTP/1.1" 200
```

← → ↻ ⓘ localhost:5000

Example of Static Files Loading

Hello MVSR, Javascript loaded

Click

6.3 Handling forms through html methods

AIM: To design a flask application that creates a form to reverse a string using get and post methods.

PROCESS:

- a) Create a folder “flaskdemos” and open it in VS Code IDE
- b) Write the below python code and save it as “strform.py”
- c) Open terminal and type below command to start Flask Server
 - i) For Ubuntu
python3 strform.py
 - ii) For Windows
python strform.py
- d) Open browser and type below Url to see the output
localhost:5000

PROGRAM:

```
from flask import Flask,request
app=Flask(__name__)
@app.route("/",methods=['GET','POST'])
def rev():
    if request.method=='POST':
        n=request.form["nm"]
        print("post invoked")
        r=n[::-1]
        return f"<h3>Reverse of a given String {n}={r}</h3>"
    else:
        print("get invoked")
        return '''
            <h3>A Flask Example to Demonstrate GET,POST Methods</h3>
            <form method="POST">
                Enter a string to reverse:
                <input type="text" name="nm"><br><br>
                <input type="submit" value="Reverse">
            </form>
        '''
app.run()
```

OUTPUT:

```
D:\PythonFlask>python strform.py
* Serving Flask app 'strform'
* Debug mode: off
WARNING: This is a development server. Do not use it in a p
.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
get invoked
127.0.0.1 - - [30/Nov/2025 23:52:55] "GET / HTTP/1.1" 200 -
post invoked
127.0.0.1 - - [30/Nov/2025 23:53:41] "POST / HTTP/1.1" 200 -
```

← → ↻ ⓘ localhost:5000

A Flask Example to Demonstrate GET,POST Methods

Enter a string to reverse:

← → ↻ ⓘ localhost:5000

Reverse of a given String MVSR=RSVM

6.4 Demonstration of Template Engine: jinja2

AIM: To design a flask application that demonstrates template engine:jinja2

PROCESS:

This application requires Two files:

i) tablerender.py

ii) tabledict.html

- a) Create a folder “flaskdemos” and open it in VS Code IDE
- b) Write the below python code and save it as “templateengine.py”
- c) Create a folder “templates” inside “flaskdemos”
- d) Create a new file “tabledict.html” and save it in the “templates” folder.
- e) Write the below codes and save them in appropriate folders.
- f) Open terminal and type below command to start Flask Server

i) For Ubuntu

python3 templateengine.py

ii) For Windows

python templateengine.py

- g) Open browser and type below Url to see the output

localhost:5001

PROGRAM:

tablerender.py

```
from flask import Flask,render_template
app=Flask(__name__)
dict={"name":"John",
      "age":60,"job":"Manager","address":"Hyd",
      "salary":2000}
@app.route("/")
def result():
    return render_template("tabledict.html",result=dict)
if __name__=="__main__":
    app.run(debug=True,port=5001)
```

tabledict.html

```
<!DOCTYPE html>
<html>
<head>
    <title>TableDict</title>
</head>
<body>
    <h3>An Example of Template Engine:Jinja2</h3><hr/>
    <h2>Displaying Employee dictionary </h2>
    <table border="1">
        <tr><th>Employee Field</th><th>Employee Value</th></tr>
        {% for key,value in result.items()%}
        <tr><td>{{key}}</td><td>{{value}}</td></tr>
        {%endfor%}
    </table>
</body>
</html>
```

OUTPUT:

```
D:\PythonFlask>python tablerender.py
* Serving Flask app 'tablerender'
* Debug mode: on
WARNING: This is a development server
.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 336-254-196
```

← → ↻ ⓘ localhost:5001

An Example of Template Engine: Jinja2

Displaying Employee dictionary

Employee Field	Employee Value
name	John
age	60
job	Manager
address	Hyd
salary	2000

PART-VII CRUD operations using MongoDB and Flask

Using Python Flask, users can interact with backend databases like MongoDB and MySQL to perform CRUD operations.

To interact with MongoDB, python uses a built-in module “pymongo”.

“Pymongo” provides a ‘MongoClient’ object using which users connect to mongodb and invoke mongodb functions.

To use the pymongo module first we must install it using the below command.

pip install pymongo

7.1 Inserting employee records in MongoDB using Python Flask

AIM: To write a python flask program that connects to mongodb and inserts few employee records.

PROCESS:

- a) Start MongoDB Server using either Windows or Ubuntu
- b) Create a “test” database and “emp” collection.
- c) Create a folder “pymongodemo” and open the folder in VS Code IDE
- d) Write the below Python program and save it as “**dbinsert.py**”
- e) Open terminal and type below command (for ubuntu)

python3 dbinsert.py

OR (for Windows)

python dbinsert.py

- f) Open browser and type below url to see the output
localhost:5000

PROGRAM:

```
from pymongo import MongoClient
import flask
conn=MongoClient("localhost",27017)
app=flask.Flask(__name__)
db=conn.test;coll=db.emp;
r1=[{"_id":8908,"name":"shyam","salary":5000},
     {"_id":8909,"name":"ram","salary":6000},
     {"_id":8910,"name":"jayram","salary":7000}]
@app.route("/")
def insert():
    try:
        coll.insert_many(r1);
```

```
        s1=f" {len(r1)} Records inserted..check in mongodb."
        return flask.jsonify(message=s1)
    except Exception as e:
        return f"Error={e}"
    finally:
        conn.close()
app.run()
```

OUTPUT:

```
D:\PythonFlask>python dbinsert.py
* Serving Flask app 'dbinsert'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production c
.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [30/Nov/2025 21:48:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/Nov/2025 21:48:41] "GET /favicon.ico HTTP/1.1" 404 -
```

localhost:5000

The Python Tutorial... Gmail Maps XML DTD – Docume... Certificate | Pantech... MV

Pretty-print ☒

```
{
  "message": " 3 Records inserted..check in mongodb."
}
```

```
test> db.emp.find()
[
  { _id: 8908, name: 'shyam', salary: 5000 },
  { _id: 8909, name: 'ram', salary: 6000 },
  { _id: 8910, name: 'jayram', salary: 7000 }
]
test>
```

7.2 Fetching employee records in MongoDB using Python Flask

AIM: To write a python flask program that connects to mongodb and displays employee records.

PROCESS:

- a) Start MongoDB Server using either Windows or Ubuntu
- b) Connect to the “test” database and open the “emp” collection. Check if records exist or not. If not, insert a few employee records.
- c) Create a folder “pymongodemo” and open the folder in VS Code IDE
- d) Write the below Python program and save it as “**dbfetch.py**”
- e) Open terminal and type below command (for ubuntu)
python3 dbfetch.py

OR (for Windows)

python dbfetch.py

- f) Open browser and type below url to see the output
localhost:5000

PROGRAM:

```
from pymongo import MongoClient
import flask
app=flask.Flask(__name__)
conn=MongoClient("localhost",27017)
print("Mongodb connection success")
db=conn.test;coll=db.emp;
result=coll.find({})
print(result)
@app.route("/")
def fetch():
    return flask.jsonify([k for k in result])
if __name__=="__main__":
    app.run()
```

OUTPUT:

```
D:\PythonFlask>python dbdisplay.py
Mongodb connection success
<pymongo.synchronous.cursor.Cursor object at 0x00000292C13E4160>
* Serving Flask app 'dbdisplay'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [30/Nov/2025 22:04:35] "GET / HTTP/1.1" 200 -
```

← → ↻ localhost:5000 ☆ {

Pretty-print ☐

```
[{"_id":8908,"name":"shyam","salary":5000}, {"_id":8909,"name":"ram","salary":6000}, {"_id":8910,"name":"jayram","salary":7000}]
```

7.3 Updating employee records in MongoDB using Python Flask

AIM: To write a python flask program that connects to mongodb and updates a few employee records.

PROCESS:

- a) Start MongoDB Server using either Windows or Ubuntu
- b) Connect to the “test” database and open the “emp” collection. Check if records exist or not. If not, insert a few employee records.
- c) Create a folder “pymongodemo” and open the folder in VS Code IDE
- d) Write the below Python program and save it as “dbfetch.py”
- e) Open terminal and type below command (for ubuntu)
python3 dbfetch.py

OR (for Windows)

python dbfetch.py

- f) Open browser and type below url to see the output
localhost:5000

PROGRAM:

```
from pymongo import MongoClient
import flask
app=flask.Flask(__name__)
q1={};q2={}
fkey=input("Enter non-id filter key:")
fvalue=input("Enter filter value:")
q1[fkey]=fvalue
key=input("Enter update key:")
value=input("Enter update value:")
q2[key]=value
@app.route("/")
def update():
    try:
        conn=MongoClient('localhost',27017)
        print("Connection success..." +str(conn))
        db=conn.test;coll=db.emp;
        result=coll.update_many(q1,{"$set":q2})
        return f"<h3>{result.modified_count} Records
Updated..check in mongodb</h3>"
```

```
except Exception as e:  
    print("Error="+e)  
finally:  
    conn.close()  
app.run()
```

OUTPUT:

```
D:\PythonFlask>python dbupdate.py
Enter non-id filter key:name
Enter filter value:shyam
Enter update key:salary
Enter update value:8000
* Serving Flask app 'dbupdate'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
Connection success...MongoClient(host=['localhost:27017'], c
127.0.0.1 - - [30/Nov/2025 22:23:18] "GET / HTTP/1.1" 200 -
```

← → ↻ ⓘ localhost:5000

1 Records Updated..check in mongodb

```
test> db.emp.find()
[
  { _id: 8908, name: 'shyam', salary: 5000 },
  { _id: 8909, name: 'ram', salary: 6000 },
  { _id: 8910, name: 'jayram', salary: 7000 }
]
test>

test> db.emp.find()
[
  { _id: 8908, name: 'shyam', salary: '8000' },
  { _id: 8909, name: 'ram', salary: 6000 },
  { _id: 8910, name: 'jayram', salary: 7000 }
]
test>
```

7.4 Deleting employee records in MongoDB using Python Flask

AIM: To write a python flask program that connects to mongodb and deletes a few employee records.

PROCESS:

- Start MongoDB Server using either Windows or Ubuntu
- Connect to the “test” database and open the “emp” collection. Check if records exist or not. If not, insert a few employee records.
- Create a folder “pymongodemo” and open the folder in VS Code IDE
- Write the below Python program and save it as “**empdelete.py**”
- Open terminal and type below command (for ubuntu)

python3 empdelete.py

OR (for Windows)

python empdelete.py

- Open browser and type below url to see the output
localhost:5000

PROGRAM:

```
from pymongo import MongoClient
import flask
app=flask.Flask(__name__)
key=input("Enter key:")
value=input("Enter value:")
@app.route("/")
def delete():
    try:
        conn=MongoClient('localhost',27017)
        print("MongoDB Connection success...")
        db=conn.test;coll=db.emp;
        result=coll.delete_many({key:value})
        return f"<h3>{result.deleted_count} Records Deleted..check in mongodb</h3>"
    except Exception as e:
        print("Error="+e)
    finally:
        conn.close()
app.run()
```

OUTPUT:

```
D:\PythonFlask>python dbdelete.py
Enter key:name
Enter value:shyam
* Serving Flask app 'dbdelete'
* Debug mode: off
WARNING: This is a development server. Do not use it in a p
.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
MongoDB Connection success...
127.0.0.1 - - [30/Nov/2025 22:39:43] "GET / HTTP/1.1" 200 -
```

← → ↻ ⓘ localhost:5000

1 Records Deleted..check in mongodb

```
test> db.emp.find()
[
  { _id: 8908, name: 'shyam', salary: '8000' },
  { _id: 8909, name: 'ram', salary: 6000 },
  { _id: 8910, name: 'jayram', salary: 7000 }
]
test> db.emp.find()
[
  { _id: 8909, name: 'ram', salary: 6000 },
  { _id: 8910, name: 'jayram', salary: 7000 }
]
test>
```